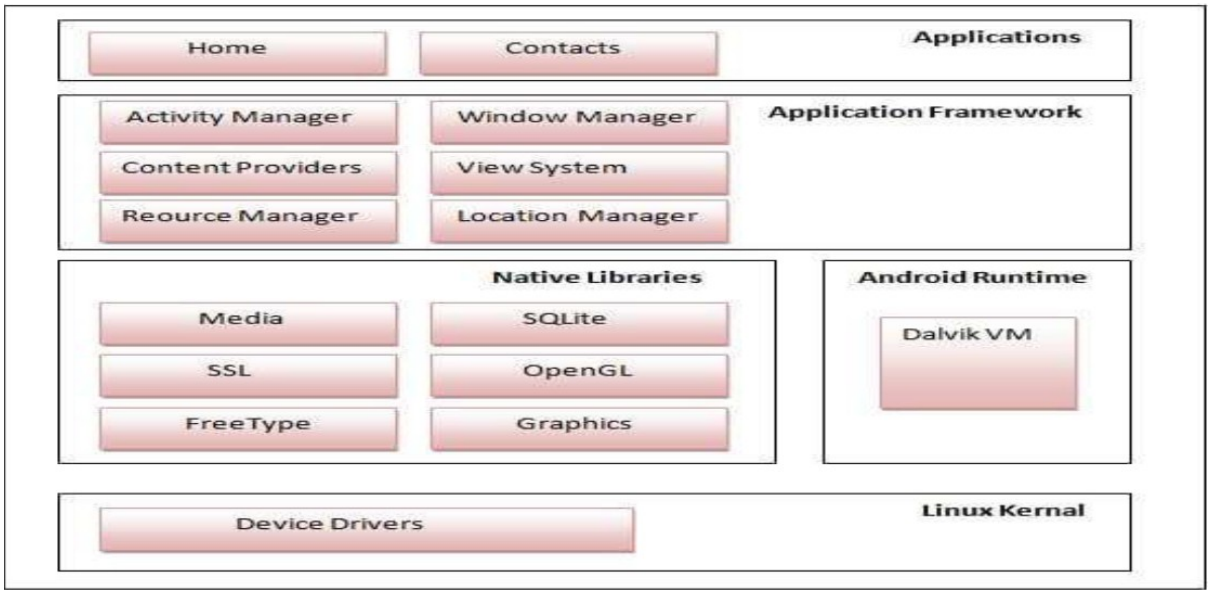
❖ Android architecture:



Android architecture is structured into five main layers, each serving a specific role in the system:

1. Linux Kernel:

- The core of the Android OS, responsible for managing hardware, security, and processes.
- Handles drivers for several device.
- Provides features like process management, memory management, and power management.

2. Native libraries:

- Provide essential functionalities to the Android system and applications.
- These libraries interact with the **Android Runtime (ART)** and **Application Framework** for performance optimization.

3. Android Runtime (ART) :

- ART is the modern runtime that executes app code efficiently.
- Replaces the older Dalvik Virtual Machine (DVM) for better performance.

4. Application Framework:

- Provides APIs for app developers to interact with system services.
- Key components include:
    - Activity Manager (controls app lifecycle)
    - Window Manager (handles UI rendering)
    - Location Manager (GPS & location services)
    - Notification Manager (alerts & notifications)

5. Applications:

- The top layer where user and system apps run.
- Includes pre-installed apps like Messages, Phone, and Camera.
- User-installed apps also operate within this layer.

❖ **System Requirements for Android Studio:**

Windows

- **OS**: Windows 10 or later (64-bit)
- **RAM**: 8 GB (16 GB recommended)
- **Storage**: 8 GB free space (SSD recommended)
- **Processor**: Intel/AMD 64-bit CPU (4+ cores recommended)
- **Graphics**: OpenGL 2.0+ support

macOS

- **OS**: macOS 12 (Monterey) or later
- **RAM**: 8 GB (16 GB recommended)
- **Storage**: 8 GB free space (SSD recommended)
- **Processor**: Apple Silicon (M1/M2) or Intel 64-bit
- **Graphics**: Metal support required

Linux

- **OS**: Ubuntu 20.04+, Debian 11+, Fedora 37+, or other 64-bit Linux distributions
- **RAM**: 8 GB (16 GB recommended)
- **Storage**: 8 GB free space
- **Processor**: 64-bit Intel/AMD CPU
- **Additional**: GNU C Library (glibc) 2.31+, OpenGL 2.0+

- **Native Platform vs Cross-Platform Development:**

Feature	Native Development	Cross-Platform Development
Definition	Apps built for a specific OS (Android/iOS)	Apps built to run on multiple OS (Android & iOS)
Languages	Java/Kotlin (Android), Swift/Objective-C (iOS)	Flutter (Dart), React Native (JavaScript), Xamarin (C#)
Performance	High	Slightly lower
Development Time	Longer	Faster
Cost	Higher	Lower
Maintenance	More effort	Easier

- **APK (Android Package) vs. AAB (Android App Bundle):**

Feature	APK (Android Package)	AAB (Android App Bundle)
Definition	A compiled package containing the entire app	A publishing format that contains all app resources but is optimized per device by Google Play
File Extension	.apk	.aab
Size	Larger	Smaller
Installation	Can be installed directly on a device	Cannot be installed directly, must be uploaded to Google Play
Distribution	Used for sideloading and third-party stores	Used only for Google Play distribution
Performance	May include unnecessary resources	Optimized, faster downloads, and smaller app size

- **What is a Layout in Android?**

A **layout** in Android defines the structure and arrangement of UI elements (Views) on the screen. It is specified using **XML**.

Types of Layouts in Android:

1. **Linear Layout:**
   - Arranges views in a horizontal or vertical.
   - Example: android:orientation="vertical"
2. **Constraint Layout:**
   - Allows positioning using **constraints**.
   - Reduces nested layouts, improving performance.
3. **Frame Layout:**
   - Holds **one** child view or overlapping views.
   - Used for **fragments**, custom views, or overlays.
4. **Table Layout:**
   - Arranges views in **rows and columns** like a table.
   - Example: TableRow for each row.
5. **Grid Layout:**
   - Similar to Table Layout but more flexible with **rows & columns**.
   - Example: android:columnCount="2"

❖ **putExtra() and getExtra() Methods in Android:**

These methods are used to **pass data between activities** using **Intents**.

1. putExtra() (Sender)

- Used to **add data** to an Intent before starting a new activity.
- Syntax:

  intent.putExtra("key", "value"); // Passing dataExample:

  intent.putExtra("username", "Kashyap");

2. getExtra() (Receiver)

- Used to **retrieve data** sent from another activity.
- Syntax:

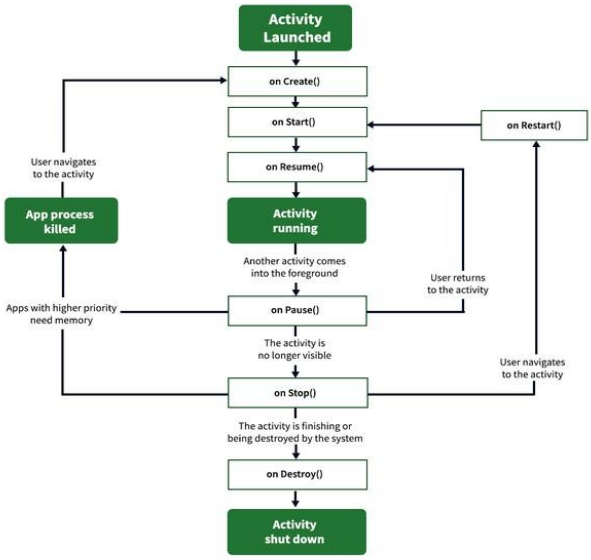
  String data = getIntent().getStringExtra("key"); // Getting data
- Example:

  String username = getIntent().getStringExtra("username");

❖ What is an Activity in Android?

An **Activity** is a single, focused **UI screen** in an Android app. It interacts with the user and displays content. Every time you see a screen (e.g., a login screen, settings screen), it's likely an Activity.

Activity Lifecycle:



Activity Lifecycle in Android

Activity Lifecycle States

1. **onCreate()** – Called when the activity is first created.
2. **onStart()** – Called when the activity becomes visible to the user.
3. **onResume()** – Called when the activity starts interacting with the user.
4. **onPause()** – Called when the activity is partially obscured or paused.
5. **onStop()** – Called when the activity is no longer visible.
6. **onRestart()** – Called when the activity is coming back to the foreground after being stopped.
7. **onDestroy()** – Called before the activity is destroyed, often used to clean up resources.

❖ Android Resource Types:

In Android development, resources are elements such as images, strings, layouts, and other assets that are external to the application code. Resources are stored in the res directory of an Android project.

Android supports various types of resources, each serving a specific purpose.

1. Drawable Resources:
➢Example: res/drawable/ic_launcher.xml
➢Drawable resources are used for define various graphics with bitmaps or XML.
➢Saved in res/drawable/ and accessed from the R.drawableclass.

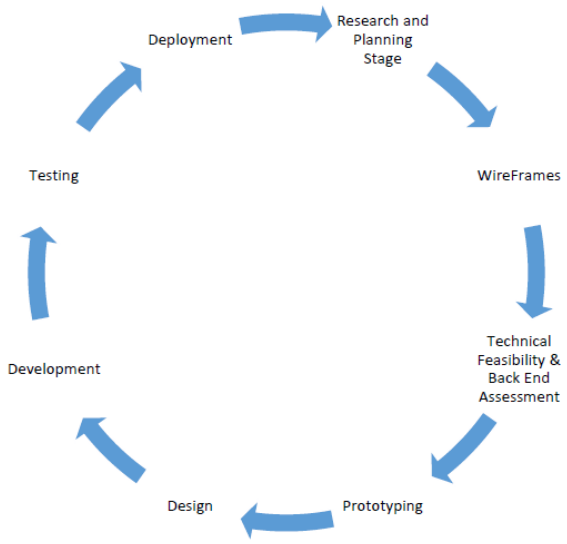
2. Layout Resources:
➢Example: res/layout/activity_main.xml
➢Layout resources define the structure and appearance of user interface.
➢Saved in res/layout/ and accessed from the R.layoutclass.

3. String Resources:
➢Example: res/values/strings.xml
➢String resources store text values, which allows for easy localization and modification of text without changing the application code.
➢Saved in res/values/ and accessed from the R.string, R.array, and R.plurals classes.

4. Color Resources:
➢Example: res/values/colors.xml
➢Color resources define color values that can be used throughout the application.
➢Saved in res/values/and accessed from the R.colorclasses.

5.DimenResources:
➢Example: res/values/dimens.xml
➢Dimenresources store dimension values such as widths, heights, and margins.
➢Saved in res/values/ and accessed from the R.styleclass.

❖ Mobile Application Development LifeCycle:-



1. Research and Planning Stage
    - Define the app's purpose and goals
    - Initial project scope and resource planning

2. Wireframes
    - Create basic layout structures for the app's UI/UX
    - Helps visualize user flow and screen transitions

3. Backend Assessment
    - Evaluate platforms needs
    - Backend infrastructure planning (APIs, databases)

4. Prototyping
    - Interactive models based on wireframes
    - Used for feedback and validation

5. Design
    - Finalize visual elements (colors, typography, icons)
    - Ensure user-friendly and aesthetic interface

6. Development
    - Frontend and backend code implementation
    - Platform-specific(Native) or cross-platform development

7. Testing
    - Performance, and security testing
    - Identify and fix bugs before release

8. Deployment
    - Publish the app on app stores
    - Ensure compliance with guidelines and store policies

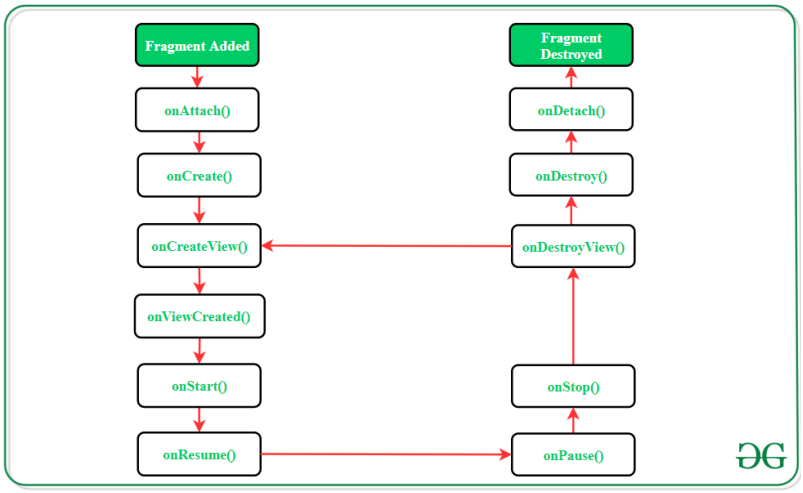
❖ What is a Fragment in Android?

A Fragment is a reusable portion of UI in an Android activity. It behaves like a mini-activity and can be combined to build dynamic and flexible UIs.

Types of Fragments:-

1. Single Fragment
   - Used in small screens (phones). One fragment shown at a time.
2. List Fragment
   - Displays a list of items (like ListView) easily.
3. Fragment Transaction
   - User can make transition among multiple fragments at run time.
   - User can switch between fragments by switching on tabs.

Fragment Lifecycle:-
The Fragment Lifecycle is closely tied to the Activity Lifecycle, but has its own distinct methods:



1. onAttach() – Fragment is attached to Activity
2. onCreate() – Initialize fragment
3. onCreateView() – Inflate UI/layout
4. onViewCreated() – Called after view creation
5. onStart() – Fragment becomes visible
6. onResume() – Fragment is now interacting with the user
7. onPause() – Not in focus, but visible
8. onStop() – Fragment is no longer visible
9. onDestroyView() – Clean up views
10. onDestroy() – Clean up fragment state
11. onDetach() – Fragment is detached from Activity

❖ What is an Intent:

An Intent is a messaging object that allows components (such as activities, services, and broadcast receivers) to communicate with each other

Android intents are mainly used to:

- Start the service
- Launch an activity
- Display a web page
- Display a list of contacts
- Broadcast a message
- Dial a phone call etc.

Types of Intent:

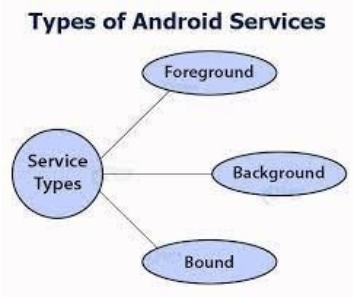
- Explicit Intent: Used when you specify the target component.
  Example: starting a specific activity or service.

- Implicit Intent: Used when you don't specify the target component directly. Instead, you declare an action, and the system determines the appropriate component.
  Example: Opening a web page in the browser.

❖ What is service:

A service is a component that runs in the background, it acts as an invisible worker of our application

Types of service:



Foreground services:

- Services that notify the user about its ongoing operations are termed as Foreground Services.
- Users can interact with the service by the notifications provided about the ongoing task. Such as in downloading a file, the user can keep track of the progress in downloading and can also pause and resume the process.

Background services:

- These services run in the background, such that the user can't see or access them.
- These services do not notify the user about ongoing background tasks and users also cannot access them

Bound Services:

- Bound service runs as long as some other application component is bound to it.

- Many components can bind to one service at a time, but once they all unbind, the service will destroy.

❖ UI vs UX comparison in table form:

Aspect	UI (User Interface)	UX (User Experience)
Definition	Visual design and layout of the product	Overall experience a user has while interacting with the product
Focus Area	Look, style, and feel (colors, typography, visuals, buttons)	Usability, flow, accessibility, and efficiency
Goal	Make the product visually appealing and consistent	Make the product useful, usable, and enjoyable
Involves	Graphic design, typography, spacing	User research, prototyping, testing
Tools Used	Figma, Adobe XD, Photoshop	Figma, Adobe XD, Axure, user testing tools
Who Works on It?	UI Designers, Graphic Designers	UX Designers, Product Designers

❖ UI widgets:

1. Button
   - A button is a clickable element that performs a specific action when pressed.
   - For example, when you click the "Submit" button, it sends your information to the server.

2. Text Field (Input Box)
   - A text field is a box where users can type.
   - For instance, when you sign in to a website, you use a text field to enter your email or password.

3. Checkbox
   - A checkbox allows users to select one or more options from a list.
   - An example would be selecting multiple items for delivery preferences.

4. Radio Button
   - A radio button lets users select only one option from a group of choices.
   - For example, when filling out a survey, you might use radio buttons to pick your age range or gender.

5. Dropdown Menu
   - A dropdown menu is a compact list that appears when the user clicks on it, allowing them to choose one item from several options.
   - A common use case is selecting a country during account registration.

❖ Event Handling:

- Events are the actions performed by the user in order to interact with the application, for e.g. pressing a button or touching the screen.
- The events are managed by the android framework in the **FIFO** manner i.e. First In – First Out. Handling such actions or events by performing the desired task is called **Event Handling.**

Event handling:

- OnClick() – This method is called when the user clicks, touches, or focuses on any view (widget) like Button, ImageButton, Image, etc.

- OnLongClick() – This method is called when the user presses and holds a particular widget for one or more seconds.

- OnMenuItemClick() – This method is called when the user selects a menu item.

- OnTouch() – This method is called either for a movement gesture on the screen or a press and release of an on-screen key.

❖ GridView

- GridView is used to display items in a two-dimensional grid (rows and columns). It's perfect for showing images or items in a tiled layout.
- Example: A photo gallery app where each image is shown in a grid format.
- **Common Attributes:**
  - android:numColumns – Number of columns (e.g., 3)
  - android:verticalSpacing – Space between rows
  - android:horizontalSpacing – Space between columns

❖ WebView

- WebView lets you display web pages inside your app using a URL or raw HTML.
- Example: Showing a website or an online article without opening a browser.
- **Common Attributes:**
  - android:layout_width / layout_height – Set size
  - android:id – To reference in code

❖ ScrollView

- ScrollView allows the user to scroll vertically through a long layout that doesn't fit on the screen.
- Example: A long form or article that needs vertical scrolling.
- Note: It contain only one child layout, usually a LinearLayout.
- **Common Attributes:**
  - android:fillViewport – Ensures content fills full screen

❖ ListView

- ListView displays a scrollable list of items vertically. It's one of the basic ways to show repeated items like texts or images.
- Example: A contacts list or settings menu.
- ⚠ Less preferred now compared to RecyclerView due to performance issues.
- **Common Attributes:**
  - android:divider – Sets a custom divider between list items
  - android:dividerHeight – Height of the divider

❖ RecyclerView

- RecyclerView is an advanced and flexible version of ListView. It's used to display large sets of data efficiently by reusing item views (recycling).
- Example: Chat messages, product lists in shopping apps.
- More customizable and better for performance compared to ListView.
- Common Attributes:
  - android:layoutManager (in code) – Set layout: linear, grid, etc.
  - android:orientation (in code) – Set scroll direction

❖ Six abstract methods:

query()	A method that accepts arguments and fetches the data from the desired table. Data is retired as a cursor object.
insert()	To insert a new row in the database of the content provider. It returns the content URI of the inserted row.
update()	This method is used to update the fields of an existing row. It returns the number of rows updated.
delete()	This method is used to delete the existing rows. It returns the number of rows deleted.
getType()	This method returns the Multipurpose Internet Mail Extension(MIME) type of data to the given Content URI.
onCreate()	As the content provider is created, the android system calls this method immediately to initialise the provider.

❖ Storages in Android

- In Android, there are 2 types of storages are available in android.
  - Data storage (Databases, SQLite, Realm, etc..).
  - File storage (SD Card, Phone Storage).

- The system provides several options for you to save your app data.
  - App-specific storage: Store files that are only used for your application, either in dedicated directories within an internal storage or different dedicated directories within external storage.
  - Shared Storage: Store files that your app intends to share with other apps, including media, documents, and other files.
  - Preferences: Store private, primitive data in key-value pairs inside root directories in xml format.
  - Databases: Store structured data in a private database using the Room persistence library.

❖ What is SharedPreferences?

SharedPreferences is used to **store small amounts of key-value data** on the device — like settings, flags, or user preferences.

 When to Use?

Use it when you need to save:

- Login state (isLoggedIn = true)
- User settings (e.g., darkMode = on)
- Simple data like username, language, etc.

 Key Characteristics:

- Stores **primitive data types only**: String, int, float, boolean, long
- Data is **persistent**, even after the app is closed
- Stored in an **XML file** in internal storage
- **Private to the app** by default

 Important Methods:

Method	Description
putString(key, value)	Store a string
getString(key, default)	Retrieve a string with default
remove(key)	Removes a specific key
clear()	Removes all key-value pairs
apply()	Saves changes **asynchronously**
commit()	Saves changes **synchronously**

 Saving Values to SharedPreferences:

We have the following steps to save values in SharedPreferences.

- **Get the SharedPreferences:** Inside your activity we will call the method getSharedPreferences().

- **Initialize the Editor:** initialize the editor where we will put the values to be saved, in key-value pairs.

- **Put the values:** We have the method putDataType() inside the editor.

- **Apply the changes:** simply call the apply() method to save the changes.

❖ What is Internal Storage?

Internal storage in Android is a **private storage space** on the device where an app can **save files** (like text, images, or data files). Files stored here are **only accessible to that app**, unless the device is rooted.

 Key Features:

- **Private to the app** – Other apps cannot access these files.
- **Automatically deleted** when the app is uninstalled.
- Good for storing **secure or sensitive data**.
- Can store any file type: .txt, .json, .png, etc.
- No special permission needed to use it.

 File Types You Can Store:

- Text or JSON data
- Images, videos, or audio
- Cache files

 Modes of Internal Storage:

- MODE_PRIVATE — In private mode the data stored earlier is always overridden by the current data i.e every time you try to commit a new write to a file which removes or override the previous content.

- MODE_APPEND — In this mode the data is append to the existing content i.e keep adding data.

❖ Methods to Store Data In Android:

 getExternalStorageDirectory()

- Not Recommended (old method).
- Used to access the root of external storage.
- Not safe or recommended now.

 getExternalFilesDir(String type)

- Recommended method.
- Saves files privately for the app.
- Files are deleted when the app is uninstalled.
- Example: storing app-specific data.

 getExternalStoragePublicDirectory(String type)

- For public files accessible by all apps.
- Files are not deleted after app uninstall.
- Example: camera images, downloaded files.

❖ What are Content Providers?

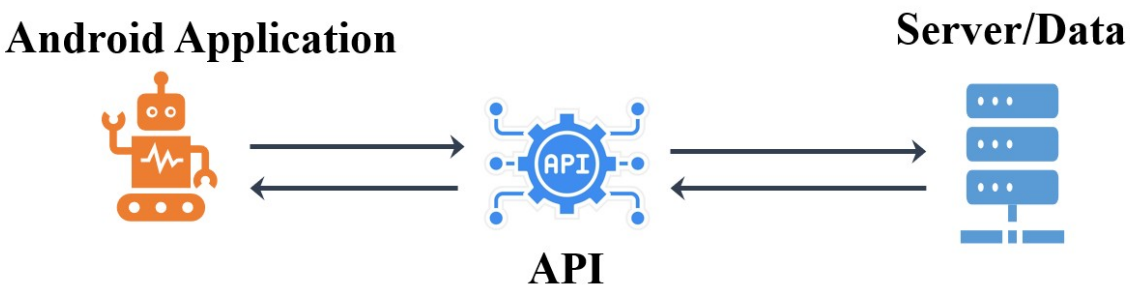
- A Content Provider component supplies data from one application to others on request.
- requests are handled by the methods of the ContentResolver class.
- Users can manage to store the application data like images, audio, videos, and personal contact information.
- A content provider can use different ways to store its data and the data can be stored in files, in a database(SQLite) or even over a network.

- **Content URIs:**
  - The most important concept to understand when dealing with content providers is the content URI. Whenever you want to access data from a content provider you have to specify a URI.

- **Operations in Content Provider:**
  - Create: Operation to create data in a content provider.
  - Read: Used to fetch data from a content provider.
  - Update: To modify existing data.
  - Delete: To remove existing data from the storage.

❖ Explain what is API and explain its type:

An **API (Application Programming Interface)** is a set of rules that allows software applications to communicate with each other. It defines **how requests should be made**, **what data formats to use**, and **how responses are delivered**.



Types of API:

- Web APIs (Most Common): Used to communicate over the internet using HTTP requests.
  - Example: RESTful APIs, SOAP APIs.
- Operating System APIs: Allows apps to interact with OS services.
  - Example: Android API, Windows API.
- Library APIs: Provides functions for specific tasks.
  - Example: OpenGL (for graphics), TensorFlow (for AI).
- Hardware APIs: Enables apps to interact with hardware.
  - Example: Camera API, Bluetooth API.

❖ Web API

- A Web API is an application programming interface which is use either for web server or a web browser.

Key concept:

- Client-Server Architecture
- Each request from the client is independent and does not rely on previous interactions.
- Use of HTTP Methods
  - GET – Retrieve data.
  - POST – Send new data.
  - PUT – Update existing data.
  - DELETE – Remove data.

Types of web API:

- RESTful API (Representational State Transfer)
  - The most common type of web API.
  - Follows REST principles: stateless, resource-based, and uses HTTP methods.
  - Example: GitHub API, Twitter API.

- SOAP API (Simple Object Access Protocol)
  - Uses XML messages for communication.
  - More secure and reliable, but complex.
  - Used in financial systems, banking applications.

- GraphQL API
  - Allows clients to request specific data instead of fetching everything.
  - Example: Facebook's GraphQL API.

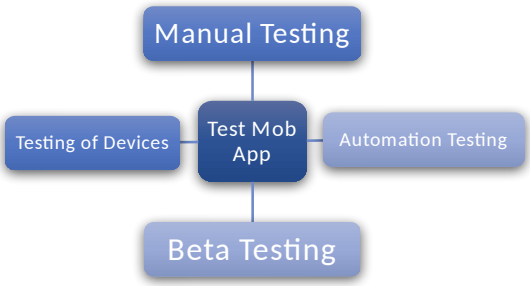
❖ **Introduction to JSON data:**

- o JavaScript Object Notation, is a format for sharing data.
- o It is a format for structuring data. This format is used by different web applications to communicate with each other.

Key Features of JSON:

- Human-readable: JSON is easy to understand and write, making it suitable for developers and non-developers alike.
- Compact format: It is a lightweight data format that enhances performance in data exchanges.
- Cross-platform support: JSON is supported all modern programming languages and platforms, including browsers, mobile apps, and servers.
- Flexible data types: JSON supports strings, numbers, arrays, objects, and null values, making it highly versatile for various use cases.
- Text-based format: JSON is a text-based format, making it simple to transmit over HTTP protocols.

❖ **Mobile application testing:**



1. Manual Testing
You check the app by using it yourself, step by step, without any tools.

2. Automation Testing
A computer program or tool tests the app for you. It's faster and good for repeating tests.

3. Beta Testing
You let real users try the app before launch to find any last bugs or issues.
Types:

- Traditional Beta testing
- Public Beta Testing
- Technical Beta Testing:
- Focused Beta Testing:
- Post-release Beta Testing:

4. Testing of Devices
You test the app on different phones and tablets to make sure it works everywhere.